

تمرین ۳ و ۲ هوش مصنوعی

پارسا دارسرای. ۱۴۰۲۰۱۲۲۶۸۱۵۶

سوال ۱

(الف)

1. شروع از S با $g(S)=0$

شاخه‌ها:

$S \rightarrow A: 4$ ☐

$S \rightarrow B: 2$ ☐

کم هزینه‌ترین مسیر فعلی: $g=2$ ($S \rightarrow B$)

2. گسترش B:

$B \rightarrow E: 2+1=3$ ☐

$B \rightarrow F: 2+5=7$ ☐

لیست مسیرهای فعال:

$S \rightarrow A (4)$ ☐

$S \rightarrow B \rightarrow E (3)$ ☐

$S \rightarrow B \rightarrow F (7)$ ☐

کمترین هزینه: $g=3$ ($S \rightarrow B \rightarrow E$)

3. گسترش E:

$E \rightarrow G: 3+3=6$ ☐

مسیر جدید: $g=6$ ($S \rightarrow B \rightarrow E \rightarrow G$)

بقیه مسیرها:

$S \rightarrow A (4), S \rightarrow B \rightarrow F (7)$ ☐

4. گسترش $S \rightarrow A (4)$:

$A \rightarrow C: 4+6=10$ ☐

$A \rightarrow D: 4+3=7$ ☐

مسیرهای فعال:

$S \rightarrow B \rightarrow E \rightarrow G (6)$ ☐

$S \rightarrow A \rightarrow D (7)$ ☐

$S \rightarrow B \rightarrow F (7)$ ☐

$S \rightarrow A \rightarrow C (10)$ ☐

چون به G رسیدیم با هزینه‌ی 6 و هیچ مسیر باز دیگری با هزینه کمتر از 6 وجود ندارد، بهینه‌ترین مسیر پیدا شد.

نتیجه: Branch and Bound

مسیر $S \rightarrow B \rightarrow E \rightarrow G$:

هزینه کل 6:

ترتیب گسترش (S, B, E, A, G): پایان در (G)

(بخش ب)

تابع ارزیابی: $f(n)=g(n)+h(n)$

جدول heuristic:

S:6, A:4, B:2, C:6, D:2, E:1, F:5, G:0

خلاصه نهایی

مسیر بهینه $(S \rightarrow G): S \rightarrow B \rightarrow E \rightarrow G$

هزینه کل مسیر: 6

ترتیب گسترش گره‌ها: S, B, E, G

لاگ مرحله‌به‌مرحله (هر گسترش با g, h, f)

توضیح: وقتی می‌گوییم «open snapshot» یعنی وضعیت صف باز (گره، g, h, f) قبل از گسترش آن گره ثبت شده است.

1. گسترش S

- $g(S)=0, h(S)=6, f(S)=6$
- قبل از باز شدن S، $(S, g=0, h=6, f=6)$ open = [
- از S دو فرزند تولید شد:

▪ با $A \Rightarrow f=8, h=4, g=4$

▪ با $B \Rightarrow f=4, h=2, g=2$

2. گسترش B (کمترین $f=4$)
- $g(B)=2, h(B)=2, f(B)=4$
 - open قبل از باز شدن $B: [(B, g=2, h=2, f=4), (A, g=4, h=4, f=8)]$

از B دو فرزند تولید شد:

- $E \text{ با } g=3, h=1 \Rightarrow f=4$
 - $F \text{ با } g=7, h=5 \Rightarrow f=12$
3. گسترش E ($f=4$ — هم‌قیمت با B اما B قبلاً باز شده)
- $g(E)=3, h(E)=1, f(E)=4$
 - open قبل از باز شدن $E: [(E, g=3, h=1, f=4), (A, g=4, h=4, f=8), (F, g=7, h=5, f=12)]$

از E فرزند G تولید شد:

- $G \text{ با } g=6, h=0 \Rightarrow f=6$
- گسترش G (بعدی با کمترین f در open)
- $g(G)=6, h(G)=0, f(G)=6$
 - open قبل از باز شدن $G: [(G, g=6, h=0, f=6), (F, g=7, h=5, f=12), (A, g=4, h=4, f=8)]$

چون G هدف است الگوریتم متوقف می‌شود و مسیر بازسازی می‌شود: $S \leftarrow B \leftarrow E \leftarrow G$.

4. وقتی G با $f=6$ کشف شده، هیچ عنصر در صف باز با $f > 6$ وجود نداشت، پس تضمین بهینگی برقرار است (A^* با heuristic داده‌شده در این اجرا مسیر بهینه را یافت).

بخش ج)

- همیشه کم‌هزینه‌ترین گره (بر اساس f) را گسترش می‌کنیم.
- اگر اضافه‌سازی فرزند منجر به عبور از حد حافظه شود، بدترین گره مرز (بالاترین f ، اگر تساوی، عمیق‌ترین حذف می‌شود و مقدار f حذف‌شده در والد مربوطه به‌عنوان «بهترین جایگزین فرزند حذف‌شده» نگهداری می‌شود (backup).
- وقتی گره هدف در حافظه و قابل انتخاب برای گسترش شود و هیچ کاندیدای با f کمتر در حافظه نباشد، الگوریتم خاتمه می‌یابد.

وضعیت و لاگ مرحله به مرحله

شروع

- حافظه: { S }
- دستور: گسترش S
- open / frontier: S ($g=0, h=6, f=6$)

S مرحله 1 — گسترش

- از S فرزندان A و B تولید شدند:
 - A: $g=4, h=4 \Rightarrow f=8$
 - B: $g=2, h=2 \Rightarrow f=4$
- حافظه پس از تولید: { S, A, B } (3 گره) — زیر حد، نیازی به حذف نیست.
- ترتیب گسترش تا اکنون: S

B و گسترش f مرحله 2 — انتخاب کمترین

- frontier: B($f=4$), A($f=8$) → انتخاب B
- گسترش B → تولید E و F:
 - E: $g=3, h=1 \Rightarrow f=4$
 - F: $g=7, h=5 \Rightarrow f=12$
- حافظه موقت: { 5 } (S, A, B, E, F گره) — از حد (4) عبور کردیم → باید 1 گره حذف شود.

قانون حذف (SMA)*: حذف بدترین گره مرز (بالاترین f).

- مرزها و fها قبل از حذف: A($f=8$), E($f=4$), F($f=12$)
- بدترین: F ($f=12$) → حذف می‌شود.

بروزرسانی پس از حذف F:

- از حافظه حذف شد F
- والد F یعنی B، مقدار پشتیبان (backup) برای فرزندانش را به حداقل f موجود برای فرزندانش تنظیم می‌کند:

$$B.child_best_f = \min(f(E)=4, f(F_deleted)=12) = 4$$
- (در عمل این یعنی B اطلاع دارد اگر E کاملاً پردازش شود و نیاز به جایگزین باشد، مقدار جایگزین 12 بوده، اما با وجود E فعلاً مقدار موثر 4 است — این مقدارها برای تصمیمات بعدی استفاده می‌شوند.)

- حافظه بعد از حذف: { 4 } (S, A, B, E گره)
- ترتیب گسترش: S, B

E و گسترش f مرحله 3 — انتخاب کمترین

- frontier: E(f=4), A(f=8) → انتخاب E
- گسترش E → تولید G:
 - G: g=6, h=0 ⇒ f=6
- حافظه موقت: { 5 } (S, A, B, E, G) گره → نیاز به حذف یک گره.

مرزها قبل از حذف: (A(f=8), G(f=6) (E و B داخلی)

- بدترین: A (f=8) → حذف می‌شود (بالاترین f).

(توضیح: A هنوز باز نشده بود؛ حذفش آزادسازی حافظه را اجازه می‌دهد).

بروزرسانی پس از حذف A:

- A.child_best_f_for_A (S باید مقدار پشتیبان مربوط به آن را نگهداری کند S یعنی A حذف می‌شود. والد A = f(A_deleted) = 8).

(در آینده اگر A نیاز به بازگردانی/بازتولید شود، این مقدار به S کمک می‌کند تصمیم بگیرد).

- حافظه بعد از حذف: { 4 } (S, B, E, G) گره
- ترتیب گسترش: S, B, E

G (هدف) مرحله 4 — بررسی و گسترش

- frontier: G(f=6) در f به عنوان داخلی در حافظه باشند. کمترین E یا B و ممکن است G(f=6) است.
- وجود ندارد f < f(G) هدف است و هیچ گره دیگری در حافظه با G تولید شده و در حافظه قرار دارد؛ چون G می‌تواند خاتمه دهد و مسیر بهینه را برگرداند SMA*، ((فرصت بهتر از 6 نیست

بازسازی مسیر هدف

- رسیدن: S → B → E → G: مسیر بازسازی S از B از E از طریق G
- هزینه کل g(G)=6 (همان که در تولید G داشتیم).

SMA* خلاصه نتایج

- مسیر نهایی $(S \rightarrow G)$: $S \rightarrow B \rightarrow E \rightarrow G$
- هزینه کل مسیر: 6
- ترتیب گسترش گره‌ها: S, B, E (در نهایت G دریافت و خاتمه)
- حذف‌ها (دلایل و مقادیر):
 - حذف F هنگام گسترش B (زیرا بعد از تولید فرزندان حافظه پر شد) — F دارای $f=12$ بود (بدترین)؛ B مقدار پشتیبان فرزندان خود را به 4 (از E) نگه داشت.
 - حذف A هنگام گسترش E (زیرا تولید G حافظه را پر کرد) — A دارای $f=8$ بود؛ S مقدار پشتیبان برای A را برابر 8 نگه داشت.
- نتیجه بهینگی: با این اجرای SMA* (حافظه ۴) الگوریتم مسیر بهینه مسئله را یافت (همان مسیر و هزینه‌ای که Branch-and-Bound و A^* یافته بودند).

(سوال ۲)

(الف) $h(n) - h^*(n) = c$

چون $h(n) = h^*(n) + c$ داریم $f(n) = g(n) + h(n) = f^*(n) + c$.

افزودن یک ثابت یکسان فقط همه f ها را به‌طور مساوی زیاد می‌کند و ترتیب گسترش گره‌ها تغییر نمی‌کند.

پس تعداد گره‌های گسترش‌یافته و مسیر بهینه همانند استفاده از h^* است.

(ب) $h(n) + h^*(n) = c$

در این حالت $h(n) = c - h^*(n)$ و بنابراین $f(n) = g(n) + c - h^*(n)$.

این رابطه ترتیب گره‌ها را برعکس می‌کند و هئورستیک دیگر قابل‌قبول نیست.

در نتیجه A^* ممکن است گره‌های بیشتری گسترش دهد و حتی جواب غیر بهینه بدهد.

(ج) مقایسه

- (الف): بهینگی و کارایی حفظ می‌شود.
- (ب): بهینگی از بین می‌رود و جستجو کندتر و نامطمئن می‌شود.

سوال ۳)

عکس ها
مقایسه:

همه روش ها جواب مسئله را پیدا می کنند، اما کمیت کار (تعداد تخصیص ها و بازگشت ها) به شدت وابسته به میزان استنتاج داخلی است.

Forward Checking نسبت به **Backtracking** پایه ای سود قابل توجهی می دهد.

Full Lookahead / MAC بهترین کارایی (کمترین جستجو) را دارد ولی هزینه هر گام (اجرای AC) بالاتر است؛ در مسائل بزرگتر یک trade-off بین هزینه استنتاج و صرفه جویی در جستجو وجود دارد.

سوال ۴)

۱) Node consistency

هیچ قید یک متغیره (unary) داده نشده؛ بنابراین همه دامنه ها در ابتدا **node-consistent** هستند (هیچ مقدار به خاطر قید تک متغیره حذف نمی شود).

۲) Arc consistency (قابلیت سازگاری قوس ها، AC)

ما قوس ها را تکراراً بررسی و دامنه ها را اصلاح می کنیم:

- بررسی قوس $A \rightarrow B$ (برای هر $a \in D(A)$ باید $\exists b \in D(B)$ که $b > a$)
 - $a=6 \rightarrow b=7$ دارد $\Rightarrow$ نگه داریم.
 - $a=7 \rightarrow b > 7$ در $D(B)$ نیست $\Rightarrow$ حذف 7 از $D(A)$.
 - $a=8 \rightarrow b > 8$ نیست $\Rightarrow$ حذف 8 از $D(A)$.

$\Rightarrow D(A) = \{6\}$ حالا

- بررسی قوس $B \rightarrow A$ (برای هر $b \in D(B)$ باید $\exists a \in D(A)$ که $b > a$)
 - با $D(A) = \{6\}$:
 - $b=6 \rightarrow$ نیاز به $a < 6$ که وجود ندارد $\Rightarrow$ حذف 6 از $D(B)$.
 - $b=7 \rightarrow a=6$ پشتیبانی می کند $\Rightarrow$ نگه داشته می شود.

$\Rightarrow D(B) = \{7\}$ حالا

- حالا قوس $B \rightarrow C$ با $D(B)=\{7\}$ باید $c < 7$ وجود داشته باشد $\Rightarrow c=8$ در $D(C)$ پشتیبانی می‌کند.

- قوس $C \rightarrow B$ برای هر $c \in D(C)$ باید $c > b$ وجود داشته باشد:
 - $\rightarrow c=7$ نیاز به $b < 7$ ولی $c=7 \Rightarrow D(B)=\{7\}$ فاقد پشتیبان $\Rightarrow$ حذف 7 از $D(C)$.
 - $\rightarrow b=7 \Rightarrow c=8$ پشتیبان دارد $\Rightarrow$ نگه داشته می‌شود.

$\Rightarrow D(C)=\{8\}$ حالا

پس از پایا شدن AC، دامنه‌ها به صورت نهایی هستند:

$$D(A)=\{6\}, \quad D(B)=\{7\}, \quad D(C)=\{8\}.$$

(هیچیک از قوس‌ها بیش از این باعث حذف نشد — سازگاری قوس برقرار شد.)

۳ / Graph consistency (وجود حل)

بله — پس از اعمال AC یک تخصیص یکتا بدست آمد: $A=6, B=7, C=8$ ، که هر دو قید $B > A$ و $C > B$ را ارضا می‌کند. بنابراین گراف (CSP) سازگار است و حل وجود دارد (در واقع فقط همین یک حل).

(سوال ۵)

عکس